PROJET WEB — PHP

MON PASSERELDEX

DOCUMENTATION

Application web de consultation de Pokémon

Federico Zuffa
Classe Passerelle
2026

Table des matières

Table des matières **2**
Introduction **3**
Étude d'opportunité **3**
I — Pourquoi un Pokédex en PHP 3
II — Description de l'existant 3
III — Les plus du projet 4
Analyse fonctionnelle **5**
I — Description globale 5
Lexique de l'application 5
Objectif de l'application 5
Principes d'utilisation 5
Analyse organique **6**
I — Description globale 6
II — Description des fonctions 6
fonctions/api.php — Appels à PokéAPI 6
fonctions/utils.php — Fonctions utilitaires 6
includes/header.php — En-tête commune 7
includes/filtres.php — Filtres 7
index.php — Contrôleur principal 7
favoris.php — Page favoris 7
III — Structure des fichiers 7
Tests et protocoles **8**
Améliorations possibles **9**
Conclusion **10**
Remerciements **10**
Bibliographie **10**

Introduction

Un Pokédex est un objet utilisé par les protagonistes de l'univers Pokémon ainsi que par les joueurs des jeux éponymes afin de pouvoir consulter et répertorier tous les pokémons fascinants rencontrés sur le chemin. Comme dit le proverbe, ATTRAPEZ LES TOUS !

Mon Projet Pokédex est une application web développée en PHP qui permet de consulter, rechercher et organiser les Pokémon en exploitant les données de l'API publique PokéAPI (pokeapi.co). Ce projet constitue une introduction concrète à la consommation d'API REST depuis un serveur web, une pratique aujourd'hui incontournable dans le développement de toute application moderne.

Les API REST (Representational State Transfer) sont des interfaces qui permettent à deux systèmes de communiquer via le protocole HTTP. Dans ce projet, PHP interroge PokéAPI pour récupérer dynamiquement les données de chaque Pokémon : statistiques, types, sprites, chaînes d'évolution. Cette architecture client-serveur est identique à celle utilisée dans des applications réelles telles que des dashboards, des applications mobiles ou des sites e-commerce qui consomment des services tiers.

Ce projet représente donc bien plus qu'un simple Pokédex : il constitue une introduction aux notions fondamentales du développement web moderne. L'appel HTTP, la désérialisation JSON, la gestion des états via sessions PHP, la séparation du code en modules réutilisables, autant de compétences directement transférables à n'importe quel projet professionnel s'appuyant sur des services externes.

Étude d'opportunité

I — Pourquoi un Pokédex en PHP

Si le Pokédex est un sujet ludique, il représente surtout un excellent support pour aborder des notions techniques concrètes et réutilisables dans de nombreux projets web :

- **Consommation d'API externe :** PokéAPI propose un endpoint REST documenté et gratuit retournant des données JSON. Apprendre à l'interroger depuis PHP avec file_get_contents() ou cURL, puis à désérialiser les réponses, est une compétence fondamentale du développeur web.
- **Gestion de session :** Le système de favoris repose sur $_SESSION, permettant de persister des données entre plusieurs pages sans base de données. C'est la base de tout système d'authentification ou de panier.
- **Architecture modulaire :** La séparation du projet en fichiers de fonctions (api.php, utils.php) et en includes (header.php, filtres.php) introduit les bonnes pratiques de séparation des responsabilités, prélude à des architectures MVC plus avancées.
- **Dynamisme côté serveur :** La génération de filtres par type, par génération, l'affichage conditionnel des évolutions et le bouton aléatoire illustrent la puissance du PHP pour produire des pages dynamiques sans JavaScript.

II — Description de l'existant

Il existe plusieurs catégories de solutions similaires :

- **a) Sites web officiels et encyclopédies :** Des sites comme Bulbapedia ou Serebii proposent des bases de données très complètes, mais sont statiques et ne permettent pas de personnalisation ni d'interaction côté utilisateur.
- **b) Applications front-end :** De nombreux Pokédex existent en JavaScript (React, Vue) consommant PokéAPI directement depuis le navigateur. Notre projet se distingue en effectuant les appels API côté serveur (PHP), ce qui offre plus de contrôle sur les données et évite l'exposition des clés API.
- **c) Projets académiques :** La plupart des projets similaires en PHP se contentent d'afficher des données statiques ou d'interroger une base de données locale. Notre approche en consommant une API externe en temps réel est plus représentative d'un contexte professionnel réel.

III — Les plus du projet

Par rapport aux solutions existantes, ce Pokédex présente plusieurs avantages notables. L'intégration de PokéAPI en temps réel garantit des données toujours à jour sans maintenance de base de données. La gestion des favoris en session PHP permet une expérience personnalisée sans inscription ni base de données. Le système de filtres par type et par génération, entièrement côté serveur, démontre qu'une interface riche et réactive est possible sans JavaScript. Enfin, l'architecture modulaire facilite la maintenance et l'ajout de nouvelles fonctionnalités.

Analyse fonctionnelle

I — Description globale

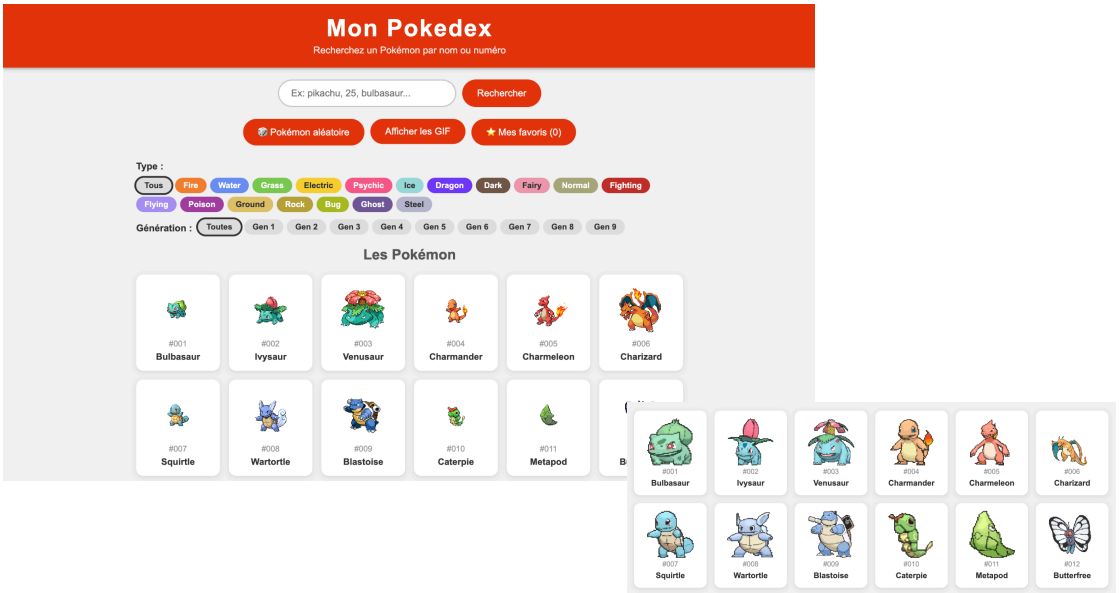
Lexique de l'application

- **Pokémon :** entité récupérée dynamiquement depuis PokéAPI, identifiée par un nom ou un numéro, mais aussi un animal imaginaire aux pouvoirs étranges nous ayant tous fait rêver enfants.
- **Sprite :** image d'un Pokémon, disponible en format PNG statique ou GIF animé.
- **Type :** caractéristique élémentaire d'un Pokémon (feu, eau, plante…), utilisée comme critère de filtre.
- **Génération :** groupe de Pokémon introduits dans une version spécifique du jeu (Gen 1 = #001 à #151, etc.).
- **Chaîne d'évolution :** séquence des formes successives d'un Pokémon, récupérée via un troisième appel API.
- **Favori :** Pokémon sauvegardé dans la session PHP de l'utilisateur pour un accès rapide.
- **Session :** mécanisme PHP ($_SESSION) permettant de conserver des données entre les pages sans base de données.

Objectif de l'application

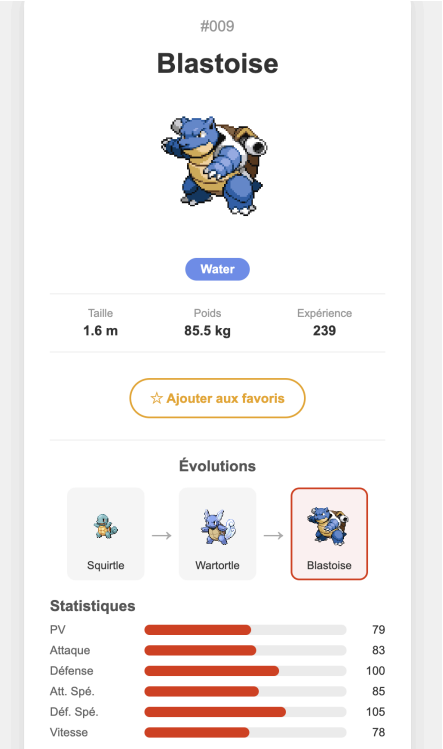
L'application met à disposition de l'utilisateur :

- Une barre de recherche permettant de trouver un Pokémon par nom ou numéro
- Une carte de détail affichant : sprite, types, taille, poids, expérience, statistiques et chaîne d'évolution
- Un bouton d'ajout/suppression aux favoris
- Un bouton de Pokémon aléatoire (redirection via rand())
- Des filtres par type (18 types) et par génération (9 générations)
- Un bouton de bascule entre sprites PNG et GIF animés
- Une grille de tous les Pokémon avec liens de navigation
- Une page dédiée aux favoris avec suppression individuelle ou globale



Principes d'utilisation

L'application fonctionne entièrement sans JavaScript. Toutes les interactions passent par des formulaires GET ou POST et des liens HTML. Chaque action (recherche, filtre, ajout aux favoris) soumet une requête au serveur PHP qui interroge PokéAPI si nécessaire, génère le HTML correspondant et le renvoie au navigateur.



Analyse organique

I — Description globale

Le projet est organisé en cinq fichiers principaux répartis dans trois couches distinctes :

Fichier	Couche	Rôle
fonctions/api.php	Modèle	Tous les appels à PokéAPI (getPokemon, getEvolutionChain…)
fonctions/utils.php	Utilitaire	Fonctions sans appel réseau (nomStat, getSpriteUrl…)
includes/header.php	Vue	Header HTML, barre de recherche, boutons secondaires
includes/filtres.php	Vue	Boutons de filtres par type et par génération
index.php	Contrôleur	Point d'entrée principal, logique métier, affichage
favoris.php	Contrôleur	Page de gestion des favoris en session

II — Description des fonctions

fonctions/api.php — Appels à PokéAPI

- **getPokemon($nom) :** Appelle /api/v2/pokemon/{nom} et retourne un tableau associatif PHP (json_decode). Retourne null si le Pokémon est introuvable (code HTTP 404).
- **getPokemonSpecies($id) :** Appelle /api/v2/pokemon-species/{id} pour récupérer les métadonnées de l'espèce, notamment l'URL de la chaîne d'évolution.
- **getEvolutionChain($url) :** Appelle l'URL de chaîne d'évolution retournée par getPokemonSpecies(). Retourne un arbre récursif.
- **extraireEvolutions($chainLink) :** Parcourt récursivement l'arbre d'évolution et retourne un tableau plat de noms. La récursion s'arrête quand evolves_to est vide.
- **getItem($itemName) :** Appelle /api/v2/item/{nom} pour récupérer les données d'un objet (non utilisé dans l'interface actuelle, prévu pour une extension).

fonctions/utils.php — Fonctions utilitaires

- **nomStat($stat) :** Traduit les noms de statistiques anglais (hp, attack…) en français (PV, Attaque…) via un tableau associatif.
- **getPokemonParGeneration($generation) :** Retourne un tableau ['debut' => N, 'fin' => M] correspondant à la plage d'IDs de la génération demandée (1 à 9).
- **getSpriteUrl($numero, $format) :** Construit et retourne l'URL du sprite GitHub selon le format choisi (png ou gif). Centralise la logique évitant la duplication dans toute l'application.
- **getFormatSprite() :** Lit $_GET['sprite'] et retourne 'gif' ou 'png' (png par défaut). Point d'entrée unique pour la lecture du paramètre de format.

includes/header.php — En-tête commune

Ce fichier contient le bloc <header> HTML, la barre de recherche (formulaire GET), et les boutons secondaires (aléatoire, PNG/GIF, lien vers les favoris). Il est inclus en tête de index.php et favoris.php via require_once. Il suppose que session_start() a déjà été appelé. Les paramètres GET actifs (type, génération, sprite) sont conservés dans des champs hidden pour ne pas être perdus lors d'une nouvelle recherche.

includes/filtres.php — Filtres

Affiche les boutons de filtre par type (18 types colorés) et par génération (Gen 1 à Gen 9). Chaque bouton est un simple lien <a href> qui ajoute le paramètre correspondant à l'URL. Le bouton actif est mis en évidence par la classe CSS 'actif'. Ce fichier ne contient aucune logique PHP complexe : il lit uniquement $_GET pour déterminer le filtre actif.

index.php — Contrôleur principal

Ce fichier orchestre l'ensemble de l'application selon l'ordre suivant :

- session_start() + require_once des dépendances
- Traitement POST (ajout/suppression favoris) avec redirection PRG
- Redirection aléatoire (rand(1, 1025) + header Location)
- Inclusion de header.php et filtres.php
- Si $_GET['pokemon'] : appel API + affichage de la carte détaillée
- Affichage de la grille selon le filtre actif (type, génération, ou tous)

favoris.php — Page favoris

Page dédiée à la consultation et gestion des favoris stockés en $_SESSION. Pour chaque Pokémon favori, un appel getPokemon() est effectué pour récupérer l'ID et construire l'URL du sprite. Propose la suppression individuelle et la suppression globale, chacune suivie d'une redirection PRG.

III — Structure des fichiers

```
PokeWeb_Project/
├── index.php
├── favoris.php
├── styleClaude.css
├── fonctions/
│   ├── api.php
│   └── utils.php
└── includes/
    ├── header.php
    └── filtres.php
```

Tests et protocoles

Les tests ont été réalisés manuellement dans un environnement XAMPP (Apache + PHP) sur macOS. Pour chaque fonctionnalité, un scénario de test a été défini avec un résultat attendu et le résultat effectivement obtenu.

N°	Description	Résultat attendu	Résultat obtenu	Validation
1	Rechercher 'pikachu'	Carte détaillée avec stats	Carte affichée correctement	✅ OK
2	Rechercher un ID (ex: 25)	Même résultat que par nom	Fonctionne identiquement	✅ OK
3	Rechercher un nom invalide	Message d'erreur rouge	Message affiché	✅ OK
4	Cliquer Pokémon aléatoire	Redirection vers un Pokémon	Redirection vers ID aléatoire	✅ OK
5	Ajouter un Pokémon aux favoris	Bouton change en '⭐ Retirer'	Bouton mis à jour après redirection	✅ OK
6	Consulter la page favoris	Grille des favoris sauvegardés	Favoris affichés correctement	✅ OK
7	Supprimer un favori individuel	Pokémon retiré de la liste	Liste mise à jour	✅ OK
8	Vider tous les favoris	Liste vide + message	Session vidée, message affiché	✅ OK
9	Filtre par type 'fire'	Grille de Pokémon de type Feu	Grille filtrée correctement	✅ OK
10	Filtre par génération 1	Pokémon #001 à #151	Plage affichée correctement	✅ OK
11	Passer en mode GIF	Sprites animés dans la grille	GIFs chargés	✅ OK
12	Chaîne d'évolution Bulbasaur	Bulbasaur → Ivysaur → Venusaur	Chaîne affichée avec sprites	✅ OK
13	Chaîne d'évolution Eevee	Eevee + 8 évolutions	Toutes les évolutions affichées	✅ OK
14	Rechargement page après favori	Pas de re-soumission du formulaire	Redirection PRG fonctionnelle	✅ OK

Améliorations possibles

Ce projet est fonctionnel et constitue une bonne base, mais de nombreuses améliorations pourraient l'enrichir significativement.

- **Mise en cache des réponses API :** Actuellement, chaque visite effectue de nouveaux appels HTTP à PokéAPI. L'implémentation d'un cache fichier ou d'un cache en base de données réduirait considérablement les temps de chargement.
- **Pagination de la grille :** Charger 1025 Pokémon en une seule requête est très lent. Un système de pagination (paramètre GET 'page') avec les paramètres limit et offset de PokéAPI permettrait un affichage instantané.
- **Persistance des favoris :** Les favoris actuels sont perdus à la fermeture du navigateur (session). Une persistance via cookies (setcookie()) ou une base de données SQL permettrait de les conserver.
- **Affichage des capacités :** PokéAPI retourne la liste des moves d'un Pokémon. Un onglet dédié dans la carte de détail permettrait de les consulter.
- **Comparaison de Pokémon :** Une page permettant de mettre deux Pokémon côte à côte avec leurs statistiques en barres pour comparer visuellement leurs profils.
- **Internationalisation :** PokéAPI retourne les noms dans plusieurs langues. Il serait possible d'afficher les noms en français en filtrant les entrées de languages[] sur 'fr'.
- **Système de jeu :** Avec tout ce que propose l'API de PokéAPI, il aurait été possible de créer un petit jeu simulant des combats pokémon, mais plus de connaissances et d'expérience avec PHP, voir l'utilisation d'un autre langage de programmation (JS) aurait étés nécessaires.

Conclusion

Ce projet m'a permis de transformer un sujet familier et ludique en une véritable application web dynamique, tout en maîtrisant des notions directement transférables à un contexte professionnel.

Sur le plan technique, la consommation de l'API REST PokéAPI est le cœur de ce projet. Chaque affichage, chaque carte, chaque filtre repose sur une ou plusieurs requêtes HTTP effectuées par PHP, dont les réponses JSON sont désérialisées et exploitées pour construire dynamiquement le HTML. Cette architecture est exactement celle d'une application consommant un service météo, un service de paiement (Stripe, PayPal) ou une API de cartographie (Google Maps). Les API REST sont aujourd'hui omniprésentes, commencer à maîtriser leur consommation depuis le serveur, c'est maîtriser la pierre de voûte fondamentale du web moderne.

La gestion des sessions PHP pour les favoris illustre quant à elle le mécanisme de persistance d'état sans base de données, point de départ de tout système d'authentification ou de gestion de panier e-commerce. L'architecture modulaire en fonctions et includes préfigure les architectures couramment utilisées dans des frameworks modernes et utilisées pour la production.

En somme, ce Pokédex n'est pas qu'un annuaire de créatures fictives : c'est une introduction concrète et motivante aux fondamentaux du développement web back-end. Les portes qu'il ouvre (authentification, base de données, cache, API privées, frameworks PHP) sont celles que tout développeur web franchira dans sa carrière. C’est aussi un petit accomplissement personnel que d’avoir pu implémenter une application fonctionnelle basé sur un univers ayant bercé mon enfance.

Remerciements

Je tiens à remercier chaleureusement l'équipe d'enseignement pour nous avoir accompagnés dans la découverte du développement web PHP. Je remercie également mes camarades pour leurs échanges et leur aide tout au long du projet.

Bibliographie

PokéAPI — Documentation officielle de l'API

https://pokeapi.co/docs/v2

MDN Web Docs — Référence PHP et HTTP

https://developer.mozilla.org

PHP.net — Documentation officielle PHP

https://www.php.net/manual/fr/

PokeAPI GitHub — Sprites des Pokémon

https://github.com/PokeAPI/sprites

Documentation PHP (filtres)
https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/presentation-filtre/#:~:text=Les%20filtres%20en%20PHP%20vont,qu'on%20va%20pouvoir%20utiliser

Andy's Tech Tutorials - "Pokemon API Tutorial | For Beginners "

Pokemon API Tutorial | For Beginners

Elk SoftWare - "How to get all data from PokéApi v2 - PHP Tutorial"

How to get all data from PokéAPI v2 - PHP Tutorial

rioredwards - "Coding Challenge : PHP Pokedex"

Coding Challenge: PHP Pokedex

ProgrammingKnowledge - "PHP REST API TUTORIAL (STEP BY STEP)"
(playlist de tutos ytb)

https://www.youtube.com/playlist?list=PLS1QulWo1RIYWjdoEC1WbT8W3XGGWVXfW

Aide au débogage et relecture de code : Claude (Anthropic) — aucun code généré